

DL-06-T8

Refactor Recipe RAG into an extensible LLM agent with structured MCP tool calling and validation

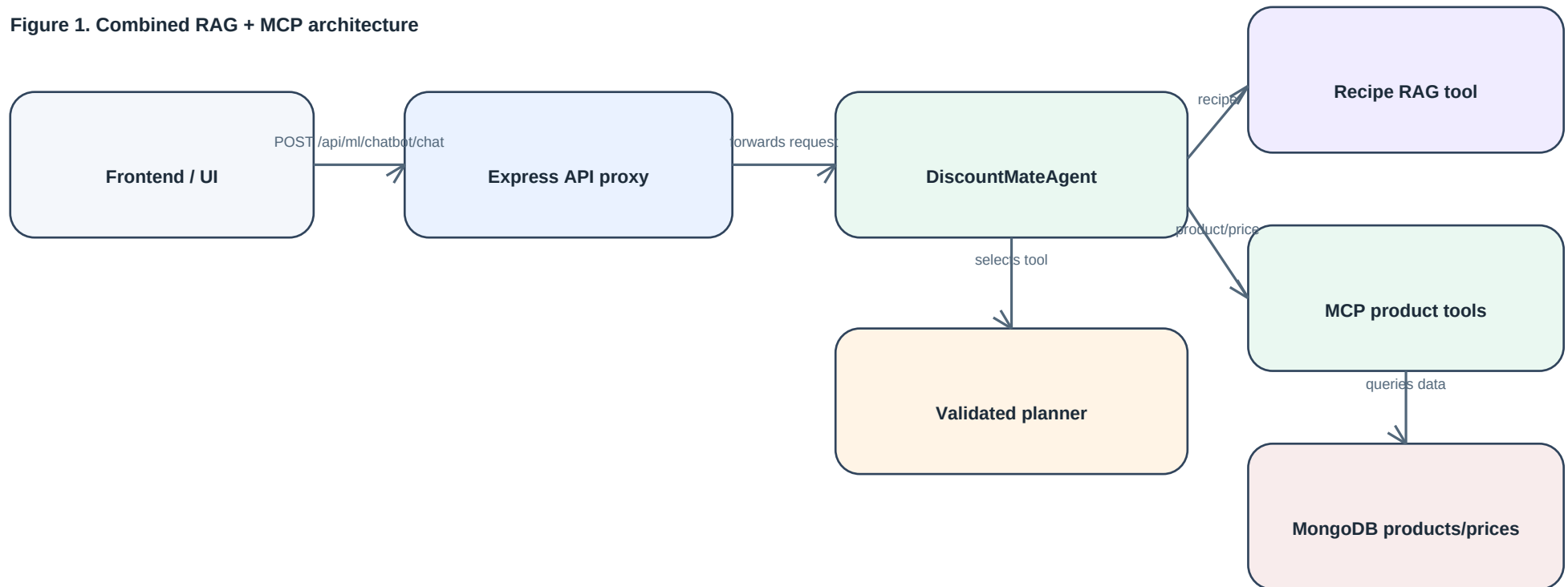
This report documents the combined chatbot implementation completed in Backend/ml-service/chatbots. The result keeps the existing Recipe RAG pipeline, exposes it as a tool, and adds a unified agent that can call either Recipe RAG or MCP-style product tools through validated contracts.

Area	What changed
Unified endpoint	Added POST /api/ml/chatbot/chat through Express and POST /api/chatbot/chat in Flask.
Recipe RAG	Wrapped the existing RecipeRAG.chat flow as the recipe_chat MCP-style tool.
MCP tools	Kept product search, details, current prices, and compare prices behind TOOL_REGISTRY.
Agent	Added DiscountMateAgent to plan, validate, execute one selected tool, and return a common response.
Validation	Added Pydantic schemas for chatbot request, tool call, agent response, and recipe tool arguments.

Combined Architecture

The frontend no longer needs to know whether a message is a recipe question or a product question. It sends one chatbot request. Express forwards that request to Flask. The agent validates the request, chooses a tool, and returns a consistent response envelope.

Figure 1. Combined RAG + MCP architecture

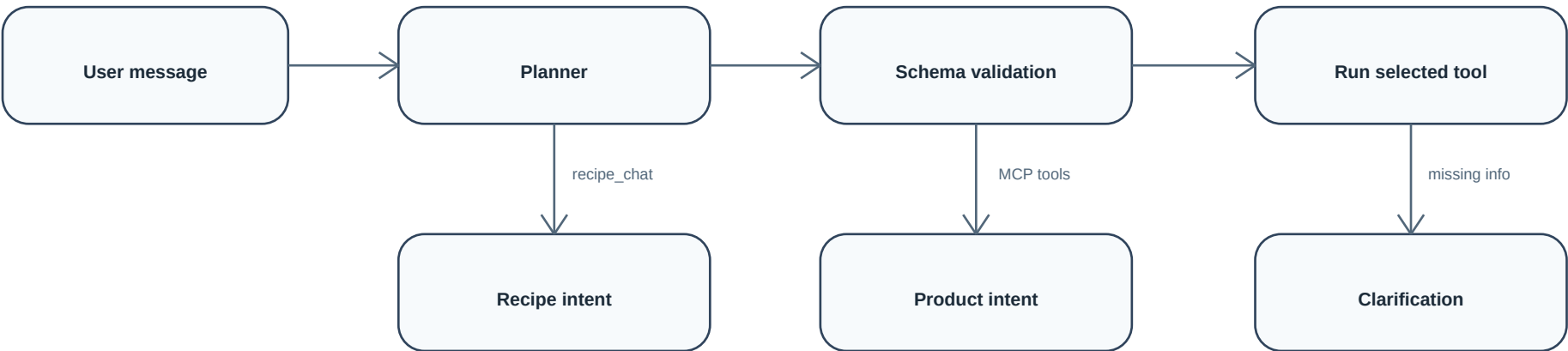


The UI sends one chatbot request. The agent validates intent and calls either Recipe RAG or MCP product tools.

Agent Responsibilities

The agent is the control layer between user messages and backend capabilities. It performs intent planning, argument completion, schema validation, tool execution, response formatting, and clarification handling.

Figure 2. Agent decision flow



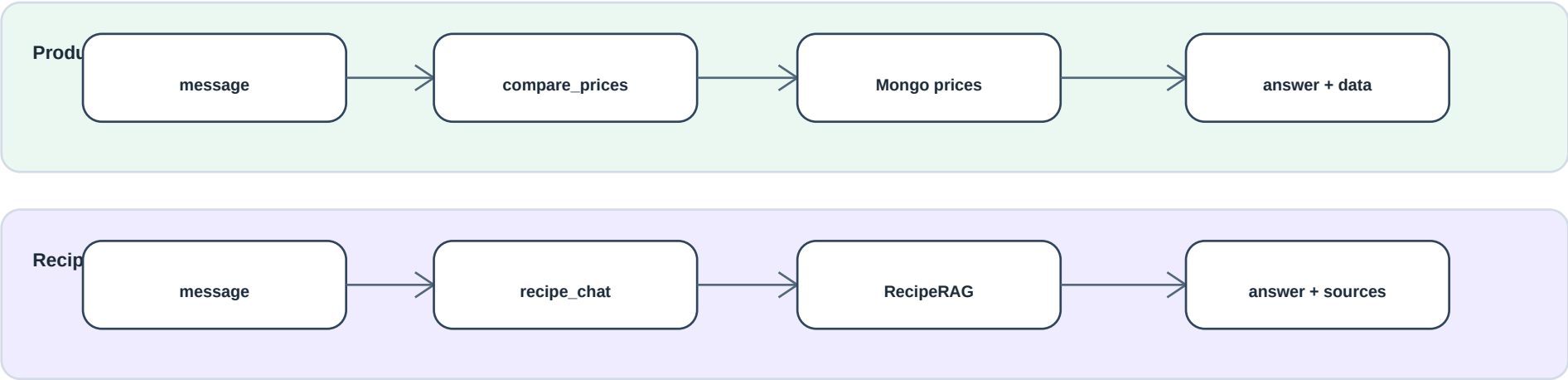
Validation rejects malformed requests, unknown tools, and invalid arguments before any tool is executed.

Function	Agent behaviour
Intent planning	Selects recipe_chat for cooking requests, compare_prices for price requests, and product tools for product lookup/details.
Argument completion	Adds session_id, top_k, context product_id, retailer filters, or product_name when safely inferred.
Validation	Validates each selected tool call with Pydantic before running it.
Execution	Runs tools only through TOOL_REGISTRY so the design remains extensible.
Clarification	Returns a structured clarification question when required data is missing.

How RAG And MCP Are Used

Recipe RAG remains responsible for retrieving recipe context and generating cooking answers from the recipe collection. MCP-style tools remain responsible for product search, product details, price retrieval, and price comparison over MongoDB product data.

Figure 3. Runtime response flow



Both paths return the same AgentResponse shape: success, answer, action, tool_calls, data, clarification, and error.

User asks	Agent action	Backend capability used	Returned data
Show me a chicken pasta recipe	recipe_chat	RecipeRAG.chat	Answer, recipe sources, product context id when products are pending.
Compare prices for Coke Zero 2L	compare_prices	MCP price comparison tool	Matched product, retailer prices, cheapest price, status.
Find milk products	search_products	MCP product search tool	Ranked product candidates.
What is the price of this product?	get_current_prices	MCP current prices tool	Current retailer prices for the context product_id.

Implementation Inventory

File	Purpose
Backend/ml-service/chatbots/agents/discountmate_agent.py	Combined agent planner, validator, executor, and response formatter.
Backend/ml-service/chatbots/schemas/agent.py	Chatbot request, tool call, and unified response contracts.
Backend/ml-service/chatbots/schemas/recipes.py	recipe_chat tool argument contract.
Backend/ml-service/chatbots/mcp_tools/recipe_chat.py	MCP-style adapter around RecipeRAG.chat.
Backend/ml-service/chatbots/mcp_tools/__init__.py	Registers recipe_chat alongside product MCP tools.
Backend/ml-service/app.py	Adds Flask /api/chatbot/chat and /api/chatbot/tools/recipe-chat.
Backend/src/controllers/ml.controller.js	Adds Express proxy handler for the combined chatbot endpoint.
Backend/src/routers/ml.router.js	Adds public /api/ml/chatbot/chat route.
Backend/ml-service/chatbots/evaluation/sample_agent_test.py	Fake-data smoke test for combined RAG + MCP routing.

Verification Performed

Check	Result
python chatbots/evaluation/sample_agent_test.py	Passed - recipe and price requests route through the combined agent.
python chatbots/evaluation/sample_tool_test.py	Passed - product MCP tool contracts still work.
python -m compileall chatbots	Passed - chatbot Python files compile.
python -m py_compile app.py	Passed - Flask app syntax is valid.
node --check src/controllers/ml.controller.js	Passed - Express controller syntax is valid.
node --check src/routers/ml.router.js	Passed - Express router syntax is valid.